ESGI · ASSEMBLEUR X86

TP 1 — Premiers pas en assembleur

Guide étudiant — cinq exercices, le code est fourni

Enseignant : **Youssef YOUNES**

Année universitaire 2026-2027

Polys générés à l'aide d'une IA — contenu relu et validé par l'enseignant.

À RETENIR
Ce TP se fait dans l'ordre, un exercice après l'autre. Chaque exercice ajoute une seule idée nouvelle et demande d'écrire entre 2 et 9 lignes. Le code complet est donné dans ce guide et dans le fichier `corrige.asm` de chaque exercice : lisez-le, comprenez chaque ligne, recopiez-le, puis vérifiez avec `make test`. Le but n'est pas de deviner, c'est de comprendre ce que fait chaque ligne. La section 4 montre, touche par touche, comment faire exécuter une seule instruction à la fois dans le débogueur et lire les registres.

1. Démarrer

Ouvrez l'environnement du cours comme à la première séance : sur GitHub, bouton Code → Codespaces, ou sur votre machine `./dev.sh` dans le dossier du dépôt. Mettez le dépôt à jour, puis allez dans le premier exercice :

```
git pull
cd exercices/tp1/ex00-observer
make check
```

La dernière ligne doit être `Environnement prêt`. Vous n'avez besoin que de trois commandes, toujours tapées dans le dossier de l'exercice :

Commande	Effet
`make test`	Compile et vérifie votre programme sur plusieurs entrées. Il faut que tout soit `PASS`.
`make run`	Compile et exécute. Vous tapez les nombres au clavier.
`make trace`	Exécute instruction par instruction et affiche la valeur des registres après chacune, comme dans le tableau du TD sur papier. C'est votre meilleur outil pour comprendre.
`make debug BREAK=nom`	Ouvre le débogueur `gdb`, arrêté à l'entrée de la fonction `nom`. C'est vous qui avancez instruction par instruction et qui demandez les registres. La section 4 montre la séance complète, touche par touche.

Dans chaque dossier : `main.c` est fourni (il lit les nombres et affiche le résultat, vous n'y touchez pas), le fichier `.asm` est le vôtre, `corrige.asm` contient la réponse, `tests/` contient les entrées et les sorties attendues.

2. Lire une ligne d'assembleur

Un processeur x86 a quelques registres : des cases de 4 octets dans lesquelles il calcule. Nous en utiliserons quatre : `eax`, `ecx`, `edx`, `esi`. Une instruction fait une seule chose, et s'écrit toujours de la même façon :

```
add  eax, [b]        ; eax = eax + b
 |    |    └── source : d'où vient la valeur (registre, nombre, ou variable)
 |    └─────── destination : où va le résultat (toujours à gauche)
 └──────────── l'opération
```

Écriture	Signification
`mov eax, 10`	eax = 10. `mov` copie, il ne « déplace » rien.
`mov eax, [a]`	eax = la valeur rangée dans la variable `a`. Les crochets veulent dire : « le contenu de la mémoire à cette adresse ».
`mov esi, tableau`	esi = l'adresse de `tableau` (sans crochets : l'adresse elle-même, pas le contenu).
`add eax, edx`	eax = eax + edx. `sub` soustrait, `imul` multiplie.
`ret`	Fin de la fonction. Le programme C récupère le résultat dans `eax`.

Une fonction en assembleur, c'est un nom suivi de deux-points (une étiquette), des instructions, et `ret` à la fin. Le `main.c` fourni appelle vos fonctions et affiche ce qu'elles ont laissé dans `eax`. Tout ce que vous avez à faire : mettre le bon résultat dans eax avant le ret.

> **À RETENIR**
> Toutes les instructions de ce TP tiennent sur la fiche réflexe (une page) distribuée avec ce guide. Gardez-la à côté de vous.

3. Exercice 00 — Observer (rien à écrire)

Dossier `exercices/tp1/ex00-observer`. Le programme est complet :

```
calcul:
    mov  eax, 10        ; eax = 10
    mov  edx, 3         ; edx = 3
    add  eax, edx       ; eax = eax + edx
    sub  eax, 5         ; eax = eax - 5
    imul eax, edx       ; eax = eax * edx
    mov  ecx, eax       ; ecx = eax
    sub  ecx, 4         ; ecx = ecx - 4
    add  eax, ecx       ; eax = eax + ecx
    ret                 ; retour : main.c affiche eax
```

Étape 1. Remplissez ce tableau à la main, une ligne à la fois, en appliquant la règle « destination = destination opération source ».

instruction exécutée	eax	ecx	edx
`mov eax, 10`			
`mov edx, 3`			

`add eax, edx`			
`sub eax, 5`			
`imul eax, edx`			
`mov ecx, eax`			
`sub ecx, 4`			
`add eax, ecx`			

Étape 2. Tapez `make trace`. La machine affiche le même tableau (avec deux colonnes de plus, `ebx` et `esi`, que l'on ignore ici) :

```
=== calcul ===
instruction (exécutée)       eax    ebx    ecx    edx    esi
mov  eax, 10                  10    ...    ...    ...    ...
mov  edx, 3                   10    ...    ...      3    ...
add  eax, edx                 13    ...    ...      3    ...
...
retour : eax =                44
```

Comparez avec votre tableau. Les valeurs de la ligne « entrée » sont les restes de ce que faisait le programme avant : un registre n'est jamais « vide », il contient toujours quelque chose.

Étape 3. `make run` affiche `calcul() = 44`. Puis changez `mov edx, 3` en `mov edx, 4`, prédisez le nouveau résultat, vérifiez.

4. Le débogueur gdb : une séance complète, touche par touche

`make trace` fait tout seul ce que vous pouvez faire à la main avec le débogueur `gdb` : exécuter une instruction, regarder les registres, recommencer. Le faire soi-même une fois, dans `ex00-observer`, c'est comprendre ce que veut dire « le processeur exécute une instruction ». Tout ce qui suit a été tapé tel quel ; les valeurs affichées sont celles que vous verrez (seules les adresses, les grands nombres en `0x8...`, peuvent changer d'une machine à l'autre).

4.1 Lancer et lire l'écran

Dans le dossier `exercices/tp1/ex00-observer`, tapez :

```
make debug
```

Cela compile, ouvre `gdb`, pose un point d'arrêt à l'entrée de la fonction `calcul`, lance le programme et le fige à cet endroit. L'écran se divise en trois zones :

En haut	**Les registres (`eax`, `ecx`, `edx`, …, `eip`, `eflags`). Un registre qui vient de changer s'affiche en surbrillance.**
Au milieu	Votre fichier `observer.asm`. La ligne surlignée est la prochaine instruction, celle qui n'a pas encore été exécutée.
En bas	La ligne de commande. Vous tapez une commande après `(gdb)` et validez avec Entrée.

En bas, vous lisez :

```
Breakpoint 1, calcul () at observer.asm:16
16      mov  eax, 10        ; eax = 10
```

Lecture : le programme est arrêté avant la ligne 16. `eax` ne vaut pas encore 10. Trois réflexes de clavier :

- Entrée sur une ligne vide répète la dernière commande (pratique pour enchaîner les `si`) ;
- Ctrl-L redessine l'écran s'il est brouillé par la sortie du programme ;
- ne tapez jamais `run` : le programme est déjà lancé par `make debug`. Sur Mac Apple Silicon, `run` échoue d'ailleurs toujours (le programme tourne dans `qemu`, `gdb` y est seulement connecté).

4.2 La séance, commande par commande

Chaque bloc ci-dessous montre ce que vous tapez (après `(gdb)`) et ce que `gdb` répond. `si` (step instruction) exécute une instruction et affiche la suivante ; `p` (print) affiche une valeur. Le nom d'un registre se tape avec un `$` devant.

```
(gdb) si
17      mov  edx, 3         ; edx = 3
(gdb) p $eax
$1 = 10
```

Le `mov eax, 10` a été exécuté, `eax` vaut 10, et la prochaine instruction est la ligne 17. Le `$1` est un numéro que `gdb` donne à chaque valeur affichée ; vous pouvez l'ignorer. Continuons :

```
(gdb) si
18      add  eax, edx       ; eax = eax + edx
(gdb) p $edx
$2 = 3
(gdb) si
19      sub  eax, 5         ; eax = eax - 5
(gdb) p $eax
$3 = 13
```

Pour voir plusieurs registres d'un coup, `info registers` suivi des noms (sans `$`). Chaque registre est affiché deux fois : en hexadécimal, puis en décimal.

```
(gdb) info registers eax ecx edx
eax            0xd                 13
ecx            0x40800d20          1082133792
edx            0x3                 3
```

`ecx` contient un grand nombre quelconque : personne n'y a encore écrit, c'est un reste de ce que faisait le programme avant. Un registre n'est jamais vide. Enchaînez maintenant cinq `si` (ou un `si` puis quatre fois Entrée) jusqu'à arriver sur le `ret` :

```
(gdb) si
20      imul eax, edx       ; eax = eax * edx
(gdb) si
21      mov  ecx, eax       ; ecx = eax
(gdb) si
22      sub  ecx, 4         ; ecx = ecx - 4
(gdb) si
23      add  eax, ecx       ; eax = eax + ecx
(gdb) si
```

```
24      ret                 ; retour : main.c affiche eax
(gdb) p $eax
$4 = 44
(gdb) p $ecx
$5 = 20
(gdb) p/x $eax
$6 = 0x2c
```

`p/x` affiche la même valeur en hexadécimal : 44 s'écrit `0x2c`. Pour voir l'instruction exactement là où le processeur est arrêté (le registre `eip`, que `gdb` appelle `$pc`) :

```
(gdb) x/i $pc
=> 0x80496f9 <calcul+25>: ret
```

Il reste à sortir de la fonction. `finish` exécute jusqu'au retour dans `main.c`, `continue` (ou `c`) laisse le programme aller jusqu'au bout, `q` quitte.

```
(gdb) finish
0x08049710 in main () at main.c:11
11      printf("calcul() = %d\n", calcul());
(gdb) p $eax
$7 = 44
(gdb) continue
calcul() = 44
[Inferior 1 (process 1) exited normally]
(gdb) q
```

> **À RETENIR**
> Après `finish`, `gdb` n'affiche pas la valeur retournée par une fonction écrite en assembleur (il ne connaît pas son type). Tapez `p $eax` : la valeur de retour est toujours là.

4.3 À vous : refaire le tableau de l'exercice 00 au débogueur

Relancez `make debug`, et après chaque `si`, tapez `info registers eax ecx edx`. Recopiez les trois nombres dans le tableau de l'étape 1. Vous devez retrouver exactement vos prédictions, puis la ligne finale `eax = 44`, `ecx = 20`, `edx = 3`. Puis changez `mov edx, 3` en `mov edx, 4` dans `observer.asm`, `make debug` à nouveau (il recompile), et vérifiez.

4.4 Les commandes à connaître

Commande	Effet
`si`	Exécute une instruction (entre dans les `call`). Entrée seule répète.
`ni`	Exécute une instruction, mais passe par-dessus un `call` sans y entrer.
`p $eax`	Affiche `eax` en décimal signé. `p/x $eax` en hexadécimal.
`info registers eax ecx edx`	Plusieurs registres d'un coup (hexa + décimal). Sans argument : tous.
`info registers eflags`	Les drapeaux positionnés par le dernier `cmp` : `ZF` (égal), `SF` (négatif)…

`p a`	La valeur de la variable `a` du fichier `.asm` (ex01 et suivants). `p &a` : son adresse.
`x/dw &a`	Un mot de 4 octets (`w`) en décimal (`d`) à l'adresse de `a`. `x/3dw $esi` : trois mots à partir de `esi`.
`x/i $pc`	L'instruction sur laquelle le processeur est arrêté.
`set $eax = 10`	Écrit 10 dans `eax`, comme `mov eax, 10`. `set var a = 7` écrit dans la variable `a` (section 4.6).
`display $eax`	Affiche `eax` automatiquement après chaque `si`. `undisplay` pour arrêter.
`break fin`	Ajoute un point d'arrêt sur l'étiquette `fin`. `info breakpoints` les liste.
`continue` ou `c`	Exécute jusqu'au prochain point d'arrêt ou jusqu'à la fin.
`finish`	Termine la fonction courante et revient dans `main.c`. Puis `p $eax`.
`Ctrl-L`	Redessine l'écran.
`q`	Quitte `gdb`.

4.5 Les trois pièges du premier jour

Piège 1 : le programme se termine sans s'arrêter. Dans `ex01` et les suivants, si vous tapez `make debug` tout court, vous lisez :

```
Function "a" not defined.
somme      = 12
...
[Inferior 1 (process 1) exited normally]
```

Le point d'arrêt par défaut est posé sur le premier nom déclaré `global` dans votre fichier. À partir de l'exercice 01, c'est la variable `a` (ou `n`), pas une fonction : `gdb` ne peut pas s'y arrêter et le programme file jusqu'au bout. Donnez toujours le nom de la fonction : `make debug BREAK=somme`. Pour choisir aussi l'entrée : `make debug BREAK=maximum IN=tests/03-a-negatif.in`.

Piège 2 : `si` sur le `ret`. Si vous faites `si` sur la ligne `ret`, vous retombez dans `main.c` (du C compilé, illisible pour l'instant). Ce n'est pas grave : `finish` ou `continue` pour en sortir, ou `q`.

Piège 3 : le programme attend au clavier. `make debug` branche automatiquement le premier fichier `tests/*.in` sur l'entrée du programme : vous n'avez rien à taper. Si rien ne se passe, c'est que vous avez tapé `run` (piège Mac) : `q`, puis relancez `make debug`.

4.6 Taper soi-même des `mov` dans le débogueur

On ne peut pas taper `mov eax, 10` directement après `(gdb)` : `gdb` n'est pas un assembleur, il répond `No symbol "mov" in current context.` Les instructions s'écrivent dans le fichier `.asm`, et `make debug` les recompile. En revanche, `gdb` sait faire la même chose qu'un `mov` ou un `add`, avec la commande `set` : écrire une valeur dans un registre ou dans une variable, pendant que le programme est arrêté. C'est le moyen de « jouer » une instruction avant de l'écrire.

Instruction dans le `.asm`	Même effet, tapé dans `gdb`	Vérification
Copier et calculer		
`mov eax, 10`	`set $eax = 10`	`p $eax` donne `10`
`mov ecx, eax`	`set $ecx = $eax`	`p $ecx` donne la valeur de `eax`
`add eax, edx`	`set $eax = $eax + $edx`	`p $eax`
`sub eax, 5`	`set $eax = $eax - 5`	`p $eax`
`imul eax, edx`	`set $eax = $eax * $edx`	`p $eax`
`neg eax`	`set $eax = -$eax`	`-3` devient `3`
`inc ecx`	`set $ecx = $ecx + 1`	`p $ecx`
`dec ecx`	`set $ecx = $ecx - 1`	`p $ecx`
Lire et écrire la mémoire		
`mov eax, [a]`	`set $eax = a`	`p $eax` donne la valeur de `a`
`mov ecx, [n]`	`set $ecx = n`	`p $ecx` donne `3` sur le 1er test
`mov esi, tableau`	`set $esi = &tableau`	`p/x $esi` donne l'adresse
`add eax, [esi]`	`set $eax = $eax + *(int*)$esi`	`p *(int*)$esi` : la case pointée
`add esi, 4`	`set $esi = $esi + 4`	`p/x $esi` a augmenté de 4
écrire `7` dans la variable `a`	`set var a = 7`	`x/dw &a` donne `7`
écrire `20` dans `tableau[1]`	`set var tableau[1] = 20`	`x/3dw &tableau` donne `1 20 3`
Comparer et sauter		
`cmp eax, edx` puis `jge ...`	`p $eax >= $edx`	`1` : le saut aurait lieu, `0` : non
`cmp eax, edx` puis `jl ...`	`p $eax < $edx`	idem
`cmp ecx, 0` puis `je ...`	`p $ecx == 0`	idem
`jmp boucle` (ou un saut pris)	`set $pc = boucle`	`x/i $pc` montre l'instruction de `boucle`
`ret`	`finish`	puis `p $eax`
Division (bonus `quotient`)		
`cdq` puis `idiv dword [b]`	`set $edx = $eax % b` puis `set $eax = $eax / b`	`17 / 5` donne `3`, reste `2`

Règles d'écriture : le nom d'un registre prend un `$` (`$eax`), le nom d'une variable n'en prend pas (`a`), et c'est le signe `=` qui joue le rôle de la virgule du `mov` : à gauche la destination, à droite la source, comme en assembleur. Quatre remarques sur les lignes moins évidentes :

- Crochets. `[a]` de l'assembleur devient simplement `a` dans `gdb` (la valeur), et `a` sans crochets devient `&a` (l'adresse). Pour une adresse qui est dans un registre, `[esi]` s'écrit `*(int*)$esi` : « l'entier de 4 octets rangé à l'adresse contenue dans `esi` ».
- `cmp` ne change aucun registre, il ne fait que préparer le saut qui suit. Dans `gdb`, on pose directement la question au processeur : `p $eax >= $edx` répond `1` (vrai : `jge` sauterait) ou `0` (faux : on continue). Pour voir ce que `cmp` a réellement laissé, `info registers eflags` (section 6).
- Un saut ne calcule rien : il change seulement la prochaine instruction, c'est-à-dire le registre `eip` (que `gdb` appelle `$pc`). `set $pc = fin_max` fait exactement ce que ferait `jge fin_max` quand la condition est vraie. Le `si` suivant repart de là.
- `idiv` fait deux résultats à la fois : le quotient dans `eax` et le reste dans `edx`. Dans `gdb`, on écrit le reste d'abord (`%`), car le quotient écrase `eax`. Attention, la division est entière et signée : `p -17 / 5` donne `-3` et `p -17 % 5` donne `-2`, comme `idiv`. Séance complète sur `ex00`, où l'on rejoue tout `calcul` à la main sans exécuter une seule instruction du programme :

```
make debug
(gdb) set $eax = 10
(gdb) set $edx = 3
(gdb) set $eax = $eax + $edx
(gdb) p $eax
$1 = 13
(gdb) set $eax = $eax - 5
(gdb) set $eax = $eax * $edx
(gdb) p $eax
$2 = 24
(gdb) set $ecx = $eax
(gdb) set $ecx = $ecx - 4
(gdb) set $eax = $eax + $ecx
(gdb) p $eax
$3 = 44
```

On retrouve 44 : c'est la preuve que votre tableau de l'étape 1 est juste. `p` sert aussi de calculatrice : `p 10 + 3` affiche `13`, `p/x 255` affiche `0xff`.

Deux remarques. D'abord, ce que vous mettez dans un registre avec `set` est écrasé dès que le programme exécute une instruction qui écrit dans ce registre : après `set $eax = 100` puis `si` sur `mov eax, 10`, `p $eax` redonne `10`. Ensuite, dans `ex01`, `set var a = 100` puis `set var b = 1` juste à l'entrée de `somme` change les nombres lus par le programme : `continue` affiche `somme = 101`, `difference = 99`, `produit = 100`, sans avoir relancé ni retapé quoi que ce soit. C'est le moyen le plus rapide d'essayer d'autres entrées.

> **À RETENIR**
> Résumé : lire un registre, c'est `p $eax` ; écrire dedans, c'est `set $eax = ...` ; exécuter la vraie instruction du programme, c'est `si`. Un `mov` tapé dans le fichier `.asm` et un `set` tapé dans `gdb` produisent le même état des registres ; seul le premier fait partie du programme.

5. Exercice 01 — Calculs

Dossier `ex01-calculs`. Le `main.c` lit deux entiers `a` et `b` et les range dans deux variables définies dans votre fichier :

```
section .data
global a, b
```

```
a:  dd 0            ; un entier de 4 octets (dd = "define double word")
b:  dd 0
```

Vous écrivez trois fonctions de deux lignes. La seule chose à comprendre : `[a]` avec crochets, c'est la valeur de `a`.

```
section .text
global somme, difference, produit

somme:                      ; eax = a + b
    mov  eax, [a]           ; eax = a
    add  eax, [b]           ; eax = eax + b
    ret

difference:                 ; eax = a - b
    mov  eax, [a]
    sub  eax, [b]
    ret

produit:                    ; eax = a * b
    mov  eax, [a]
    imul eax, [b]           ; multiplication signée
    ret
```

`make test` doit afficher 4 `PASS`. Avec `make run`, entrez `7 5` : vous obtenez 12, 2 et 35. Avec `make trace`, vous voyez les trois fonctions l'une après l'autre.

Au débogueur. Première fois qu'une fonction lit la mémoire : regardons-la. Entrée `7 5` (le premier test). Le `BREAK=somme` est obligatoire (piège 1 de la section 4.5).

```
make debug BREAK=somme
(gdb) p a
$1 = 7
(gdb) x/2dw &a
0x810b048:  7 5
(gdb) si
15      add  eax, [b]       ; eax = eax + b
(gdb) p $eax
$2 = 7
(gdb) si
16      ret
(gdb) p $eax
$3 = 12
```

`x/2dw &a` lit deux mots de 4 octets à partir de l'adresse de `a` : on voit `7` puis `5`, car `b` est rangée juste après `a` en mémoire. Pour passer directement à la troisième fonction sans refaire tous les `si` : `break produit` puis `continue` ; deux `si` plus tard, `p $eax` vaut 35. Enfin `continue` affiche les trois lignes du programme.

À RETENIR

Pourquoi `mov eax, [a]` et pas `mov eax, a` ? Sans crochets, `a` est l'adresse de la variable (un grand nombre comme 135 311 432), pas sa valeur. Essayez : retirez les crochets, `make debug BREAK=somme`, un `si`, puis `p/x $eax` et `p &a` : les deux affichent la même adresse (`0x810b048` sur l'image du cours). `make test` indique alors `somme = 135311437` au lieu de 12.

6. Exercice 02 — Comparer

Dossier `ex02-comparer`. Nouveauté : comparer deux valeurs et sauter vers une étiquette selon le résultat.

`cmp eax, edx` compare les deux valeurs sans les modifier. Juste après, un saut conditionnel utilise le résultat de cette comparaison : `jge` saute si « plus grand ou égal », `jl` si « plus petit », `je` si « égal », `jne` si « différent ». `jmp` saute toujours. Les deux instructions vont toujours par paire : un `cmp`, puis un saut.

```
maximum:                    ; eax = le plus grand de a et b
    mov  eax, [a]           ; eax = a
    mov  edx, [b]           ; edx = b
    cmp  eax, edx           ; compare a et b
    jge  fin_max            ; si a >= b : a est déjà dans eax, on saute à la fin
    mov  eax, edx           ; sinon : le maximum est b
fin_max:
    ret

absolu:                     ; eax = |a|
    mov  eax, [a]
    cmp  eax, 0             ; compare a et 0
    jge  fin_abs            ; si a >= 0 : rien à faire
    neg  eax                ; sinon : eax = -eax
fin_abs:
    ret
```

Une étiquette (`fin_max:`) est juste un nom donné à un endroit du code : « si la condition est vraie, continue à partir d'ici ». Elle doit être unique dans le fichier.

Avec `make trace IN=tests/03-a-negatif.in` (a = −3, b = 7), vous verrez que la ligne `mov eax, edx` s'exécute. Avec `tests/02-a-plus-grand.in`, elle n'apparaît pas : le saut l'a enjambée.

Au débogueur. C'est ici que `info registers eflags` devient utile : il montre ce que `cmp` a laissé et dont le saut va se servir. Avec a = −3 et b = 7 :

```
make debug BREAK=maximum IN=tests/03-a-negatif.in
(gdb) si
(gdb) si
15      cmp  eax, edx
(gdb) p $eax
$1 = -3
(gdb) p $edx
$2 = 7
(gdb) si
16      jge  fin_max
(gdb) info registers eflags
eflags         0x286              [ IOPL=0 IF SF PF ]
(gdb) si
17      mov  eax, edx       ; sinon : le maximum est b
(gdb) si
fin_max () at comparer.asm:19
19      ret
(gdb) p $eax
$3 = 7
```

Le `cmp` a calculé −3 - 7 et constaté un résultat négatif : drapeau `SF` (sign) levé. `jge` ne saute donc pas, et le `si` suivant tombe bien sur `mov eax, edx`. Refaites la même séance avec `IN=tests/02-a-plus-grand.in` (a =

7, b = 3) : après le `cmp`, `eflags` n'a plus `SF` (`[ IOPL=0 IF ]`), et le `si` sur `jge` vous amène directement sur `fin_max`. La ligne 17 n'a jamais été exécutée.

7. Exercice 03 — Boucle

Dossier `ex03-boucle`. Calculer 1 + 2 + … + n. Écrivons d'abord l'algorithme en français, puis traduisons ligne à ligne :

Algorithme	Assembleur
somme = 0	`mov eax, 0`
compteur = n	`mov ecx, [n]`
tant que compteur ≠ 0 :	`boucle:` puis `cmp ecx, 0` puis `je fin`
somme = somme + compteur	`add eax, ecx`
compteur = compteur - 1	`dec ecx`
(recommencer)	`jmp boucle`
retourner somme	`fin:` puis `ret`

```
somme_1_a_n:
    mov  eax, 0          ; somme = 0
    mov  ecx, [n]        ; compteur = n
boucle:
    cmp  ecx, 0          ; compteur == 0 ?
    je   fin             ; oui : on sort de la boucle
    add  eax, ecx        ; somme = somme + compteur
    dec  ecx             ; compteur = compteur - 1
    jmp  boucle          ; on recommence
fin:
    ret                  ; résultat dans eax
```

Une boucle, c'est un saut en arrière (`jmp boucle`) et une porte de sortie (`je fin`). Lancez `make trace` avec n = 5 et suivez `eax` et `ecx` tour après tour : 5, 9, 12, 14, 15. Puis `make trace IN=tests/03-zero.in` : avec n = 0, on sort tout de suite, la boucle ne tourne pas.

Au débogueur. Dans une boucle, taper `p $eax` à chaque pas devient lassant. `display` demande à `gdb` d'afficher un registre tout seul après chaque `si` :

```
make debug BREAK=somme_1_a_n
(gdb) display $eax
(gdb) display $ecx
(gdb) si
13      mov  ecx, [n]        ; compteur = n
1: $eax = 0
2: $ecx = 135311348
(gdb) si
boucle () at boucle.asm:15
15      cmp  ecx, 0          ; compteur == 0 ?
1: $eax = 0
2: $ecx = 5
```

Appuyez ensuite sur Entrée à répétition : à chaque passage sur `dec ecx`, `ecx` descend, et à chaque `add eax, ecx`, `eax` monte : 5, 9, 12… Pour aller directement à la sortie de la boucle sans compter les pas, posez un point d'arrêt sur l'étiquette `fin` :

```
(gdb) break fin
Breakpoint 2 at 0x8049785: file boucle.asm, line 21.
(gdb) continue
Breakpoint 2, fin () at boucle.asm:21
21      ret                 ; résultat dans eax
1: $eax = 15
2: $ecx = 0
(gdb) continue
somme de 1 a 5 = 15
```

`ecx` vaut 0 et `eax` vaut 15 : c'est exactement l'état où le `je fin` a enfin sauté. Si vous oubliez `dec ecx`, ce point d'arrêt n'est jamais atteint : `Ctrl-C` interrompt le programme, et `p $ecx` montre un compteur qui ne bouge pas.

> **À RETENIR**
> Si vous oubliez `dec ecx`, le compteur ne descend jamais et le programme tourne sans fin. `make test` affiche alors `délai dépassé (boucle infinie ?)`.

8. Exercice 04 — Tableau

Dossier `ex04-tableau`. C'est la même boucle que l'exercice 03, avec une idée en plus : une adresse dans un registre.

Un tableau en mémoire, ce sont des cases de 4 octets les unes après les autres. `esi` contient l'adresse de la case courante ; `[esi]` est le contenu de cette case ; `add esi, 4` passe à la suivante.

```
adresse    tableau     tableau+4   tableau+8   tableau+12
contenu    tableau[0]  tableau[1]  tableau[2]  tableau[3]
               ↑ esi au départ
```

```
somme_tableau:
    mov  eax, 0         ; somme = 0
    mov  esi, tableau   ; adresse = tableau   (SANS crochets : l'adresse de la 1re
case)
    mov  ecx, [n]       ; compteur = n        (AVEC crochets : la valeur de n)
boucle:
    cmp  ecx, 0         ; compteur == 0 ?
    je   fin            ; oui : on sort
    add  eax, [esi]     ; somme = somme + contenu de la case pointée par esi
    add  esi, 4         ; case suivante (un entier = 4 octets)
    dec  ecx            ; compteur = compteur - 1
    jmp  boucle         ; on recommence
fin:
    ret
```

Avec `make trace HEX=1`, vous voyez `esi` avancer de 4 en 4 (en hexadécimal, les adresses se lisent mieux). Avec `make test`, les 4 tests doivent passer, y compris le tableau vide et les nombres négatifs.

Au débogueur. L'intérêt de `gdb` ici : lire la mémoire pointée par `esi` avec la commande `x`. Entrée `3` puis `1 2 3` (premier test). Trois `si` pour passer les initialisations, puis :

```
make debug BREAK=somme_tableau
(gdb) si
(gdb) si
(gdb) si
boucle () at tableau.asm:17
17      cmp  ecx, 0         ; compteur == 0 ?
(gdb) p/x $esi
$1 = 0x810b04c
(gdb) x/3dw $esi
0x810b04c:  1 2 3
(gdb) x/3dw &tableau
0x810b04c:  1 2 3
(gdb) p n
$2 = 3
```

`esi` contient la même adresse que `&tableau`, et `x/3dw $esi` affiche les trois cases qui s'y trouvent. Faites un tour de boucle (quatre `si`, jusqu'à revenir sur `dec ecx`) et regardez de nouveau :

```
(gdb) p $eax
$3 = 1
(gdb) p/x $esi
$4 = 0x810b050
(gdb) x/dw $esi
0x810b050:  2
```

`esi` a avancé de 4 (`0x...4c` puis `0x...50`), et la case qu'il désigne contient maintenant 2, le prochain nombre à additionner. Si vous avez écrit `mov esi, [tableau]` par erreur, `p/x $esi` affiche `0x1` (la première valeur) et `x/dw $esi` répond `Cannot access memory at address 0x1` : c'est la Segmentation fault expliquée.

> **À RETENIR**
> Les deux lignes à ne pas confondre : `mov esi, tableau` (l'adresse, sans crochets) et `add eax, [esi]` (le contenu, avec crochets). Si vous écrivez `mov esi, [tableau]`, esi contient la première valeur du tableau au lieu de son adresse, et `[esi]` ira lire n'importe où : `Segmentation fault`.

9. Erreurs fréquentes

Message ou symptôme	Cause et solution
`make: *** No rule to make target`	Vous n'êtes pas dans le dossier de l'exercice : `cd exercices/tp1/ex01-calculs`.
`symbol 'fin' redefined`	Deux étiquettes portent le même nom. Renommez-en une (`fin_max`, `fin_abs`).
`undefined reference to 'somme'`	Le nom de la fonction est mal orthographié, ou la ligne `global somme, ...` a été modifiée. Majuscules et minuscules comptent.

`invalid combination of opcode and operands`	Deux opérandes en mémoire (`add [a], [b]`) : interdit. Passez par un registre : `mov eax, [a]` puis `add eax, [b]` .
`FAIL ... délai dépassé`	Boucle infinie : il manque `dec ecx` , ou le `cmp ecx, 0` / `je fin` est après le `jmp` .
`Segmentation fault`	Vous avez lu la mémoire à une mauvaise adresse : crochets en trop ou en moins (`mov esi, [tableau]`). `make debug BREAK=...` puis `continue` : `gdb` s'arrête sur l'instruction fautive, `p/x $esi` montre l'adresse absurde.
`Function "a" not defined.` puis le programme se termine	`make debug` sans `BREAK=` : le point d'arrêt est posé sur une variable. Relancez avec `make debug BREAK=somme` (section 4.5).
Dans `gdb` , rien ne se passe après `run`	Il ne faut pas taper `run` : le programme est déjà lancé. `q` , puis `make debug` à nouveau.
Résultat faux seulement avec des négatifs	Vous avez utilisé `ja` / `jb` (non signés) au lieu de `jg` / `jl` (signés), ou `mul` au lieu de `imul` .

10. Et après

Chaque `corrige.asm` se termine par un exercice supplémentaire, avec son code : `quotient` , `signe` , `factorielle` , `maximum_tableau` . Pour l'ajouter, il faut aussi déclarer la fonction dans `main.c` et l'afficher avec un `printf` . Au TP 2, dans les mêmes conditions, vous manipulerez des chaînes de caractères octet par octet, appellerez une fonction depuis une autre avec `call` , et écrirez à l'écran par un appel système : c'est le dossier `exercices/tp2-chaines` . Ce n'est qu'au TP 3 que vos fonctions recevront leurs arguments par la pile, comme en C (`exercices/tp3-fonctions`).